Redistribution status

☐ Allowed ☒ Not allowed

Seat no. _____

Course

CPE334

Software Engineering

Examination date

27 October 2026

Examination time

13.00 - 16.00 (3 hours)

Student category

☒ Regular

☒ International

Exam set (optional) : 1

Full name: _____ **Student ID:** _____

Department / Major: _____ **Section:** _____

Instructions

1. This 21-page examination (cover included) has **3 parts, 12 questions and 100 marks**.
   -- Part 1: Requirements and Security, Questions 1-4, 25 marks.
   -- Part 2: Project Planning and Execution, Questions 5-7, 25 marks.
   -- Part 3: UML/UI and Git Workflow, Questions 8-12, 50 marks.
   -- End of booklet: S1-S3 are extra practice, outside the 100 marks and 3 hours.
   Write your answers in this examination booklet.
2. Write your name and student ID on every page.
3. Use blue or black ink. If space is insufficient, continue on the reverse of the same question page.
4. No clarifications or corrections. If unclear, state and justify a reasonable assumption.

Conditions

1. **Calculators are not permitted.**
2. You may bring one A4 sheet. Write your **name and student ID** on it and **submit it with this booklet**. Failure to submit the sheet results in a **5-mark deduction from your examination score**.

* This examination has been reviewed by the Computer Engineering Department committee. *

Head of Department	Instructor / Course Coordinator
.....	.....
	Telephone: .....

Part 1 Requirements and Security (25 marks)

Question 1 Analyse stakeholder requirements **(8 marks)**

A university plans a workshop registration system. For each statement in 1.1-1.4: classify it as functional, non-functional, or a business/domain rule; justify the classification; explain its purpose and quality; give a verification method and expected result; then revise it or justify keeping it. Mark any proposed conditions or numerical targets as requiring stakeholder confirmation.

1.1 (2 marks) A signed-in student can view all workshops that are open for registration.

1.2 (2 marks) The registration process should be fast.

Question 1 (continued) Analyse the remaining statements using the same criteria.

1.3 (2 marks) University policy limits each student to three workshops per semester.

1.4 (2 marks) Students can cancel their registrations, and coordinators can view participant lists.

Question 2 Evaluate the WorkshopHub proposal **(7 marks)**

Separate online forms and manual consolidation cause duplicate workshop registrations and late confirmations. WorkshopHub lets students find and register for workshops and coordinators manage participant lists, aiming to reduce errors and administrative work. Release 1 covers free workshops only; payment and certificates come later. The team plans to finish before the next semester.

2.1 Evaluate all eight proposal elements **(4 marks)**

Mark each element present, partial, or missing, with evidence or reasoning from the proposal.

Element	Status and supporting evidence
Problem	
Users	
Solution	
Expected value	
Scope	
Risk	
Effort	
Evidence of completion	

2.2 Write an improved proposal **(3 marks)**

Keep the original problem and release scope. Fill the gaps; distinguish proposals from given facts.

Question 3 Role-Based Access Control **(4 marks)**

One role per account; no inheritance; default deny. Students may view published workshops and submit their own registrations. Coordinators may view published workshops, create and edit workshops, and export participant lists. No workshop-ownership restrictions apply.

3.1 Fill in every cell with Allow or Deny **(2.5 marks)**

Operation	Student	Coordinator
View published workshops		
Submit own registration		
Create a workshop		
Edit a workshop		
Export participant lists		

3.2 Is hiding the export button sufficient? **(1.5 marks)**

State the backend checks and one direct-request test with its expected result.

Question 4 Report-upload vulnerability **(6 marks)**

Browser-only PDF checks are bypassed by a student's script disguised as a PDF. The server stores it in a script-executable directory; opening its URL runs the script and reads other students' reports.

4.1 Analyse the incident **(3 marks)**

Identify the attacker, assets, attack surface, trust boundary crossed, and violated security objectives.

4.2 Give two security controls, each with a negative test **(3 marks)**

For each pair, give a relevant control, test procedure, and expected result.

Part 2 Project Planning and Execution (25 marks)

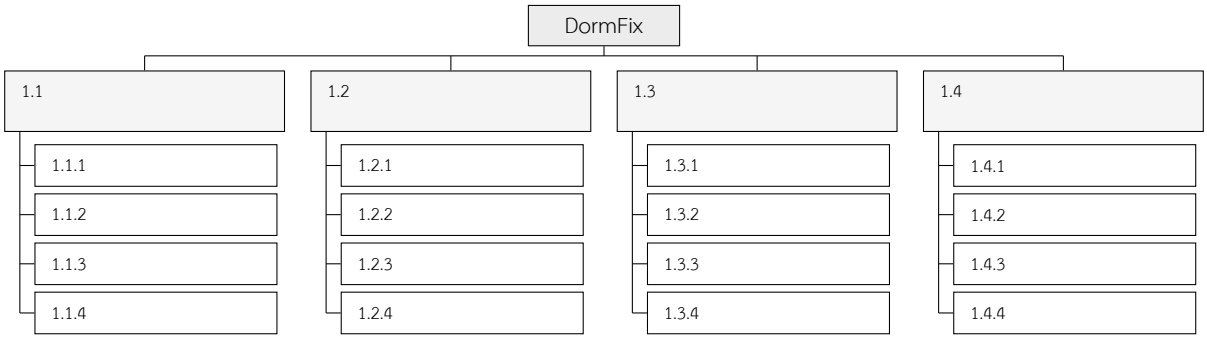
Question 5 Plan the DormFix project **(10 marks)**

You are PM of DormFix, a Waterfall dormitory maintenance-request project. Your team: Aom (BA), Bank (UX/UI), Chai (Developer), Dao (Developer/Tester). The dormitory manager is Sponsor; the head technician represents Users.

Task	Description	Owner	Days
A	Implement the approved request-form design	Chai	4
B	Get requirements approved by the dormitory manager	PM	1
C	Design screens from approved requirements	Bank	3
D	Integration-test both components	Dao	2
E	Write requirements first	Aom	3
F	Get both designs approved by the head technician	PM	1
G	Design the database from approved requirements	Chai	2
H	Get the tested system accepted by the dormitory manager	PM	1
I	Implement the approved technician-dashboard design	Dao	3

5.1 Organise the WBS by the milestones achieved **(3 marks)**

Name the milestones; order A-I, using each task once. Either order is valid for parallel tasks. Leave unused cells blank.



5.2 Complete the RACI matrix from the meeting notes **(2 marks)**

Use R, A, C, I, or "-" for no role, following these notes.

- E: Aom works; PM is accountable; Bank advises beforehand; Chai is notified afterwards; Dao has no role.
- C: Bank works; PM is accountable; Aom advises beforehand; Chai is notified afterwards; Dao has no role.
- G: Chai works; PM is accountable; Bank advises beforehand; Dao is notified afterwards; Aom has no role.
- D: Dao works; PM is accountable; Chai advises beforehand; Aom is notified afterwards; Bank has no role.

Task	PM	Aom	Bank	Chai	Dao
E					
C					
G					
D					

5.3 Build the Gantt chart **(5 marks)**

Start on Monday, 4 January 2027; work Monday-Friday, with no other holidays. Use the given owners and durations. You and all four team members work full-time; external stakeholders have other duties. Each successor starts on the working day after its predecessors and buffers finish. Add a 1-day task buffer and a 2-day project buffer only where needed. Put each milestone on the last marked day of its group's last task; put the final milestone on the last project-buffer day, with no added workday. Fill WBS No./Task/Owner/Start/Due including buffers. Use X for work, - for buffers, and * for milestones. Show dependencies (cross-panel connectors allowed), leave unused rows blank, and explain below.

Question 5.3: Gantt chart answer grids

January 2027: Weeks 1-2

No.	Task	Owner	Start	Due	4	5	6	7	8	11	12	13	14	15

January 2027: Weeks 3-4

No.	Task	Owner	Start	Due	18	19	20	21	22	25	26	27	28	29

Explanation (1 mark, included in 5.3): Justify the buffers and explain why milestones add no working time.

Question 6 Backlog grooming: badminton court booking **(10 marks)**

Issue #12, “Book a badminton court”: The client requests one-hour court bookings without queueing. Court details and opening hours are already stored; each student may hold at most one booking per day. Example: court 2 is available at 17:00 today and must display Booked after a successful booking. No assignee, milestone, or estimate is set. Choose a suitable label, use “-” for unspecified fields, and list follow-up questions separately.

6.1 Write a user story: role, function, and value **(2 marks)**

6.2 State assumptions/details supported by the client information **(2 marks)**

6.3 Write two Gherkin acceptance scenarios **(4 marks)**

Cover a successful booking based on the example, and an additional booking request when the student already has a booking that day.

6.4 Specify the issue metadata **(2 marks)**

Assignees	Labels	Milestone	Estimate

Question 7 Sprint planning: ShuttleNow **(5 marks)**

You are the Scrum Master. Sprint 6 goal: "Students can track shuttle buses and know when their bus will arrive." Capacity is 16 points, as last sprint; the backlog is in priority order. Anchor "View the list of shuttle routes" = 3 points. Scale: 1, 2, 3, 5, 8, 13, 21; larger/smaller means steps on this scale.

7.1 Estimate story points and select Sprint 6 work **(3 marks)**

Complete the Points column; mark each story Yes or No in Pick.

No.	User story	Relative to anchor	Points	Pick
1	View live bus locations	2 steps larger		
2	Pay the fare by QR code	Same size		
3	View arrival time at my stop	1 step larger		
4	Receive a bus-arrival alert	Same size		
5	View arrival times at every stop	1 step larger		
6	Fix truncated items on mobile	1 step smaller		

7.2 Total the points of the selected stories **(1 mark)**

_____

7.3 Explain each selection and exclusion **(1 mark)**

Part 3 UML/UI and Git Workflow (50 marks)

Use the following requirements for Questions 8-10.

Requirements: KU Course Registration System

R1. All users log in with a KU account through KU Single Sign-On (KUSSO), which provides role information such as Student, University Admin, and Instructor.

R2. A Department Admin can create a course by providing its course code, course name, description, credits, and fee.

R3. A Department Admin defines course offerings by selecting a semester and course code, then entering a list of sections: section number, room, class time, display notes, and maximum students. A course with four sections produces four course offerings.

R4. A University Admin can create a semester and set the currently active semester.

R5. A Student opens the Registration Form for the active semester, initially with no offerings. The system uses the logged-in username to obtain the student ID, name, degree programme, email, and login username from the KU Student Info System.

R6. A Student can add multiple offerings through the Course Offering List View. The total registration fee updates as courses are added.

R7. The Course Offering List View displays course ID, course code, course name, credits, fee, semester, section, room, class time, display notes, maximum students, number registered, and seats remaining. Students can filter offerings and select them for the form.

R8. The form starts in Draft. Students can save it and return to edit it before final submission.

R9. On submission, the system checks seat availability for every offering. If any section is full, submission fails and the student is warned to revise and resubmit. Payment proceeds only if every offering has an available seat.

R10. Payment is by credit card through KBank Payment Gateway or by bank transfer through Bangkok Bank Payment Gateway (BBLPaymentGateway).

R11. After successful payment, the form becomes Done and cannot be edited online. The student receives a registration PDF by email through KU Email (Google Workspace).

Question 8 Sequence Diagram **(10 marks)**

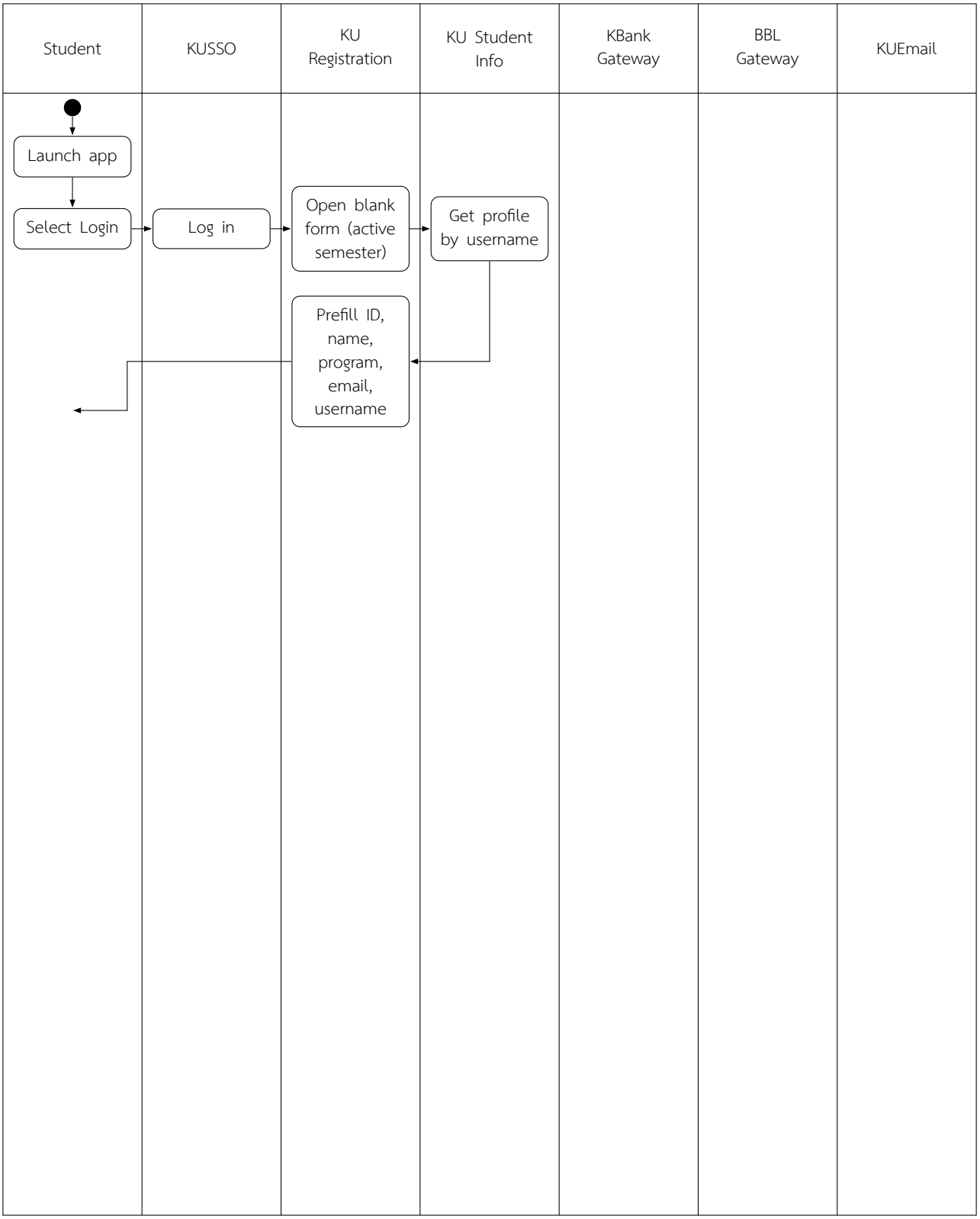
Complete the sequence diagram below; show all messages, directions, and system responses.

1. The Student logs in.
2. The system finds a Draft for the current semester containing two offerings and opens it for continuation.
3. The Student adds one more offering.
4. The Student submits, but one of the three offerings is full, so submission fails.
5. The system notifies the Student.
6. The Student replaces the full offering with another section and resubmits.
7. This submission succeeds.
8. The Student pays by bank transfer.
9. Registration is completed.



Question 9 Activity Diagram **(10 marks)**

Show one session: a Student first logs in this semester, receives a blank form prefilled with their details, selects courses, pays successfully, and gets confirmation. Include all decisions (e.g. payment method), guards, and loops (e.g. revise/resubmit until successful). Use flowchart-style decisions in the responsible swimlanes.



Question 10 Design the List View and Form View **(10 marks)**

Design generic Student interfaces for R5-R11. Mark required fields with a leading *****. Use textboxes for editable fields and diagonal hatching inside non-editable or calculated fields.

UI notation example (Invoice): Use its notation for the Registration design.

[Save] [Clear] [Close]

Invoice No. | *Invoice Date: | [Date]

*Customer Code: | [LoV] | Customer Name:

Notes:

	*Product Code	Name	*Quantity	Unit Price	Price	
1	[LoV]					+ x

Total Due:

10.1 Course Offering List View **(4 marks)**

Show all required data, filters, and offering selection; a table or cards with details is acceptable.

Question 10 (continued): Design the registration form

10.2 Student Registration Form View **(6 marks)**

Use the current semester and the signed-in Student's details. Show the header, line items, fees, controls, and how the interface appears in Draft, when an offering is full, and in Done.

Shared context for Questions 11-12

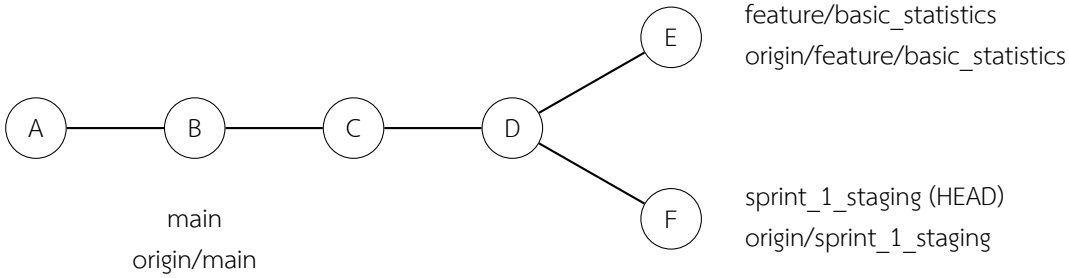
ExamScore is a Python CLI that reads names and scores from 0 to 100 from CSV, calculates statistics and the pass rate, and uses main for releases, sprint staging for integration, and feature branches for separate tasks.

Notation and assumptions

HEAD identifies the checked-out local branch. Each origin/* pointer records the latest remote state known locally. The switch command changes HEAD without moving branch pointers. Assume a clean working directory, successful commands, and no conflicts. Commands do not contact GitHub unless a fetch, push, or remote command is explicitly stated. Each question starts from its own graph. **Stage files by name; do not use `git add .`.**

Question 11 Predict the repository graph **(10 marks)**

Using the shared context, feature and sprint branches diverge from D. Assume the merge below succeeds and creates G.



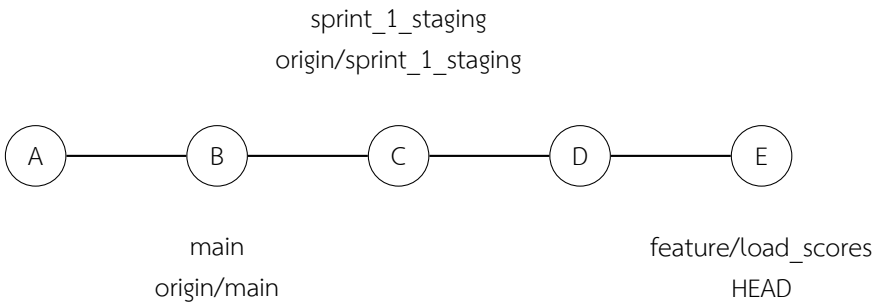

```
git merge feature/basic_statistics
git switch main
```

Draw the final graph, including both parents of G, local branches, origin/* pointers, and HEAD. Identify and justify the merge type, and explain how switch affects HEAD and the branch pointers.

Merge type, justification, and effect of switch:

Question 12 Write commands and draw the graph **(10 marks)**

Complete 12.1-12.3 consecutively. Each step starts from the result of the previous step.



12.1 Switch to sprint_1_staging and draw the graph **(2 marks)**

Command:

_____

12.2 Merge feature/load_scores into the current branch **(3 marks)**

Write the command, identify the merge type, and draw the graph after the merge.

Command and merge type:

_____

12.3 Inspect changes, stage files, and commit **(5 marks)**

After 12.2, `score_data.py` and `scores.csv` are modified and `validation_notes.txt` is new. Inspect the changes, stage only these three files, and create F with message `Document score validation cases`. Draw all pointers and HEAD.

Commands:

Graph after creating F:

Supplementary practice

S1-S3 cover question types not selected for the main paper. They are outside its 100 marks and 3-hour time limit. Questions 1-12 remain the timed mock examination.

S1 Use Case Diagram: University Grades Submission

Requirements (adapted from Lecture 4, pages 64-65)

1. Instructors log in with their KU account through KU Single Sign-On. Only valid accounts may access the system.
2. Instructors choose a semester and view the courses and sections offered in their department, supplied by the KU Registration System.
3. For a selected course and section(s), the system can load the registered students from the KU Registration System into a grade sheet.
4. A grade sheet has a header (Course, Semester) and student rows. It starts in Draft. Instructors enter or update grades and may print a copy for their records.
5. Once all grades are entered, the Instructor changes the sheet to Ready. Submission is allowed only in Ready, not Draft.
6. On submission, the system updates every student's transcript, emails each student their grade, confirms success, and changes the sheet to Submitted.

The five external systems

KU Single Sign-On	Verifies accounts and manages passwords
KU Registration System	Supplies courses, sections and registered students
KU Transcript System	Records official grades
KU Student Information System	Supplies contact details, such as student email addresses
KU Email Service (Google Workspace)	Sends grade notifications

Task: Draw a Use Case Diagram on the next page covering the Instructor's goals and the external systems. Show the system boundary, actors and associations. Use **<<include>>/<<extend>>** only where the requirements justify them. Do not replace use cases with a sequence of workflow arrows.

Brief explanation: Which steps are mandatory, which are optional, and what permits submission?

S1 (continued): Use Case Diagram workspace

University Grades Submission Supplementary; outside the 100 marks

S2 State Diagram: Service Ticket

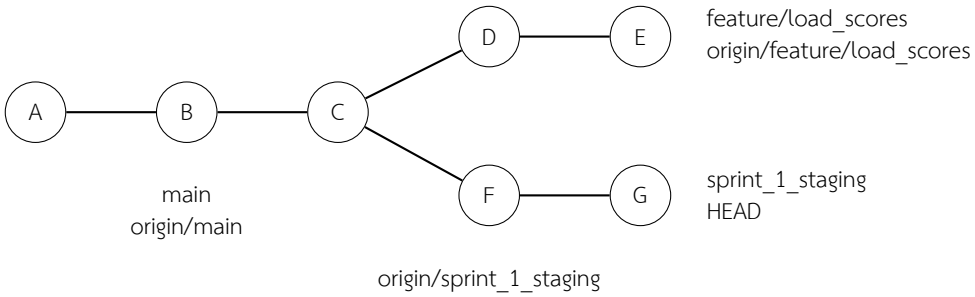
Supplementary; outside the 100 marks. Use the worked path in Lecture 4, pages 61-62, with the supporting facts on pages 45-46.

1. When a new Ticket is saved, the System automatically creates its first Action Record, sets the status to Requested, and copies Initial Due Date to New Due Date.
2. The CCE may act only on Requested tickets: approve to Approved or close to Closed by CCE, with notes or a new due date where applicable.
3. In the worked path, a Programmer starts an Approved ticket, making it In Progress. Further progress actions may record notes or a new due date while keeping In Progress.
4. When work is finished, the Programmer closes it to Closed by Programmer and sets New Due Date to the completion date.
5. Every saved action updates the Ticket's Last Status and Last Due Date from that latest action. Saved actions cannot be edited or deleted.
6. No new actions are allowed after Closed by CCE or Closed by Programmer. Customers and IT Admins do not perform these status-changing actions.

Task: Draw only the status lifecycle of one Ticket. Show initial/final nodes, states, transitions and a self-loop. Label each transition with its event and the actor in brackets, following the sample notation, e.g. `approve [CCE]`. Customer, CCE and Programmer are not states. Do not invent additional states.

S3 Git: Explain the repository graph

Supplementary; outside the 100 marks. Adapted from Samples Question 2.2, pages 8-9. Use ExamScore and the Git assumptions from Questions 11-12. Read each question independently from this snapshot unless a command is explicitly run.



S3.1 Which local branch is checked out, and which commit does HEAD identify?

S3.2 Which commit at the local sprint tip is not in the history referenced by origin/sprint_1_staging? Explain using the pointers.

S3.3 Which commit supplies the files in the current working directory?

S3.4 Can `git merge feature/load_scores` fast-forward now? What merge would be needed, and why?

S3.5 Does the graph prove that this merge would cause a file conflict? Explain.

S3.6 What may `git fetch origin` update, and what will it not move automatically?